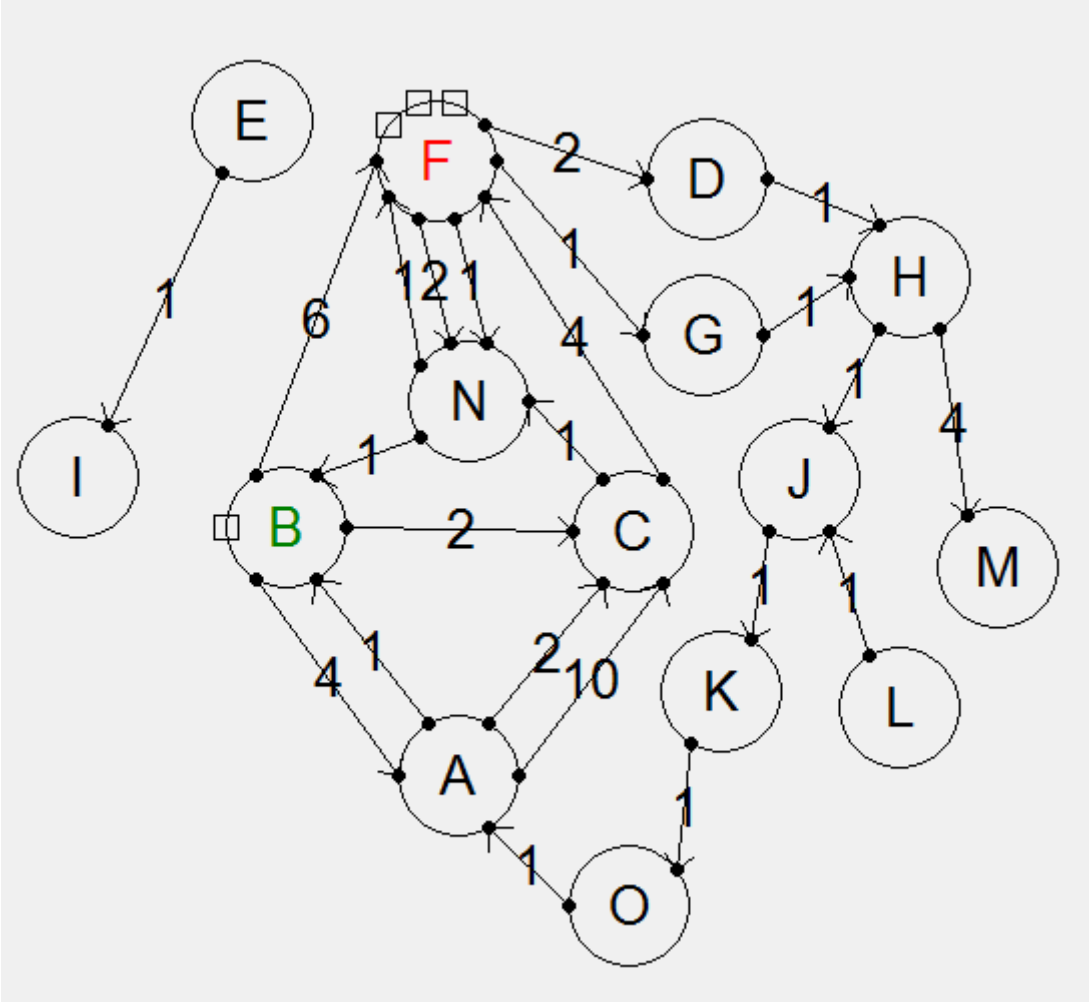
# Etude de
# Graphe orienté




Projet ISN 2ème année.

Projet réalisé par :

Luc Rosselli et Bruno Fiorucci

Version 11. Années 2014-2016

# Sommaire


I. Présentation du projet p 2

II. Prise en main de l'application p 3

III. Programmation p 6

IV. Problèmes rencontrés et améliorations possibles p 11

V. Annexes : Bibliothèques, logiciel… p 13

VI. Conclusion p 14

# I. Présentation du projet

## *Introduction*

Le projet que nous avons retenu cette année est la conception d'une application en langage Python 3. Elle permet dans un premier temps la représentation de graphes orientés. Plusieurs options seront après disponibles :

- Recherche du chemin optimal (plus courte distance) entre deux points
- Recherche des distances minimales et des trajets correspondants permettant d'atteindre tous points à partir d'un point donné par l'utilisateur.
- Transformation du graphe en arbre. Bien évidemment, les parties non connexes seront évincées.

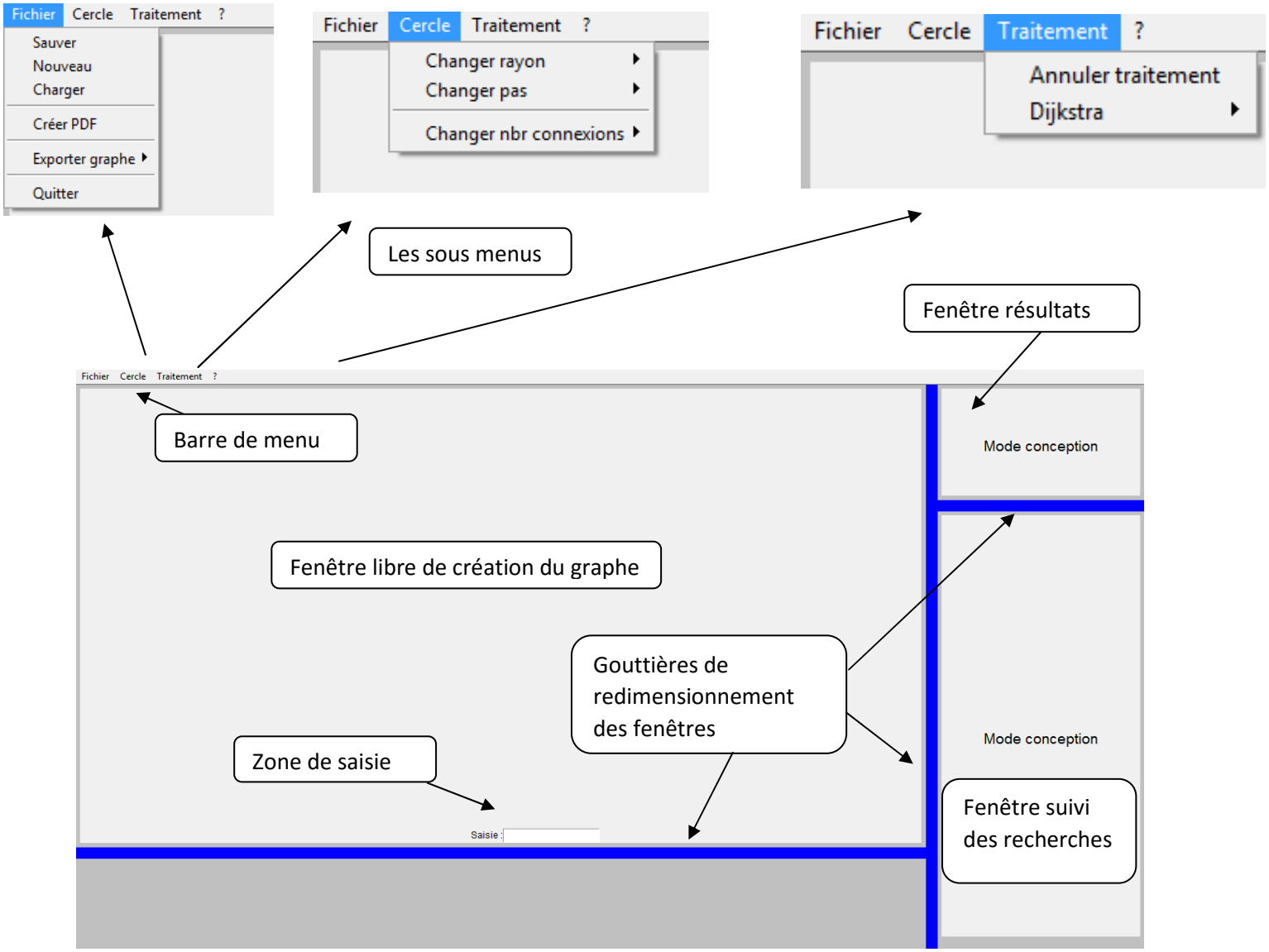
En 1ère année, nous avons élaboré un logiciel permettant d'atteindre le premier item ci-dessus. Nous avons décidé d'améliorer le code avec la mise en place d'une structure MVC et avec l'utilisation de la représentation des données par des objets.

Nous avons utilisé plusieurs structures différentes de stockage des données afin d'explorer les difficultés et les avantages de certaines options disponibles en informatique.

Enfin, nous avons mis en place un système de sauvegarde ou de visualisation externe des graphes. La dynamique étant toujours de mettre en place un maximum de dispositifs différents aux dépends d'une application ergonomique.

# II. Prise en main de l'application

## Présentation de l'interface

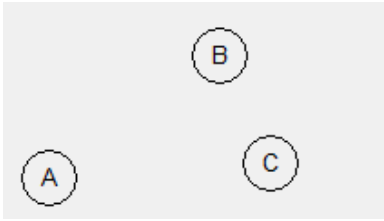



***Nota*** : Pour redimensionner les gouttières :

1. positionner le curseur de la souris sur la gouttière à déplacer.
2. Maintenir le bouton gauche de la souris enfoncé en la déplaçant.

# Présentation des différents modes :

## a) Création d'un graphe




***Création d'un nœud***

Le clic gauche dans la fenêtre de création permet l'instanciation d'un nœud.

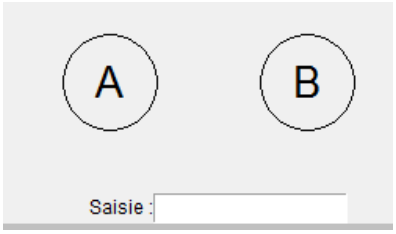
Instanciation : créer une instance de la classe Nœud.
Instance : objet de la classe.

Par défaut, les nœuds sont nommés par les lettres de l'alphabet.




La boite de saisie permet de nommer le nœud avant sa création

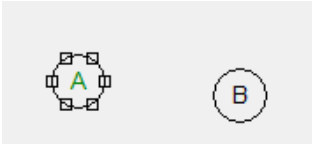
La dimension des nœuds peut être modifiée en allant dans le menu cercle.



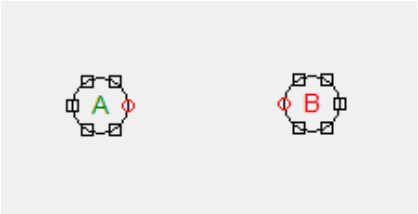

***Création d'une branche***

Le clic gauche sur un nœud permet de le sélectionner :

Le clic gauche sur un nœud sélectionné le désélectionne.

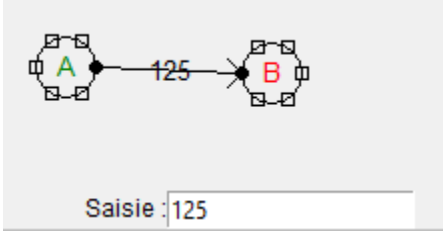



La création d'une branche s'effectue à partir de nœuds sélectionnés sur lesquels on choisit un carré de sélection comme point d'ancrage (celui-ci se transforme en un cercle rouge)




Puis on définit le poids de la branche dans la zone de saisie avant de réaliser un clic droit.

Par défaut le poids de la branche est de 1.




L'orientation de la branche est toujours du vert vers le rouge.

***Remarques*** :
Un clic droit sur un objet le détruit ainsi que tous les objets dépendants.
Glisser la souris en maintenant le bouton droit appuyé :
Si le pointeur de la souris est sur un nœud : déplacement du nœud
Sinon déplacement de tout le graphe.

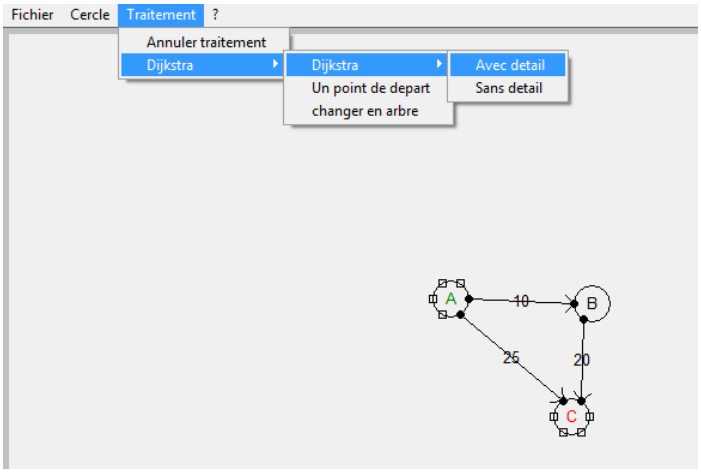
***Sauvegarde du graphe***

Une sauvegarde du graphe peut être réalisée en allant dans le menu Fichier.

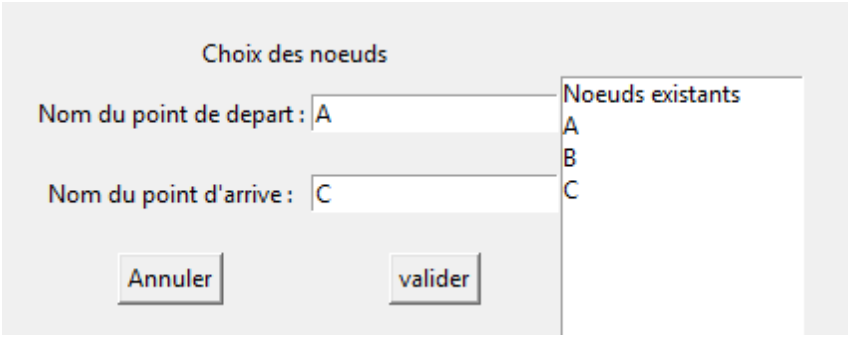
## b) Traitement

A partir d'un graphe ouvert dans la fenêtre de création, sélectionner dans le menu traitement : Dijkstra puis sélectionner avec ou sans  détail.

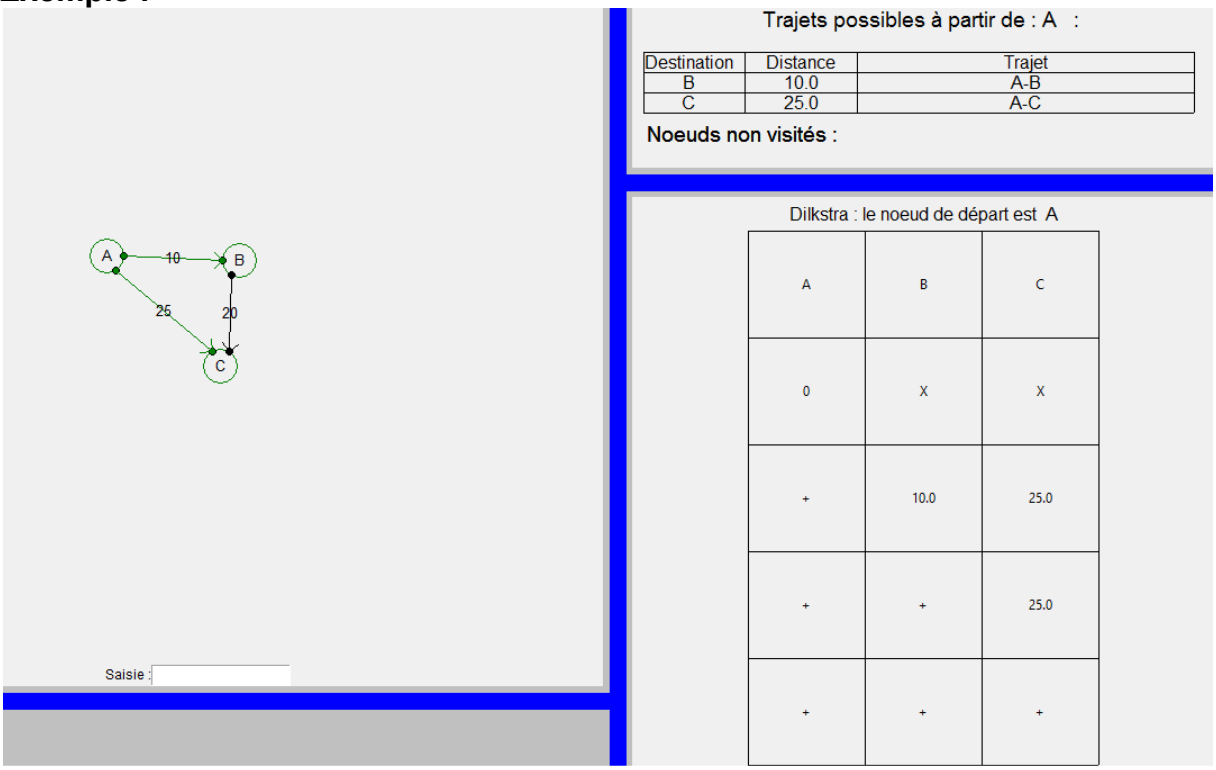
.




Dans la nouvelle fenêtre, indiquer le point de départ et le point d'arrivée puis valider




**Exemple :**



# III. Programmation

## a) Présentation de la structure MVC

Que veut-on voir à l'écran ?

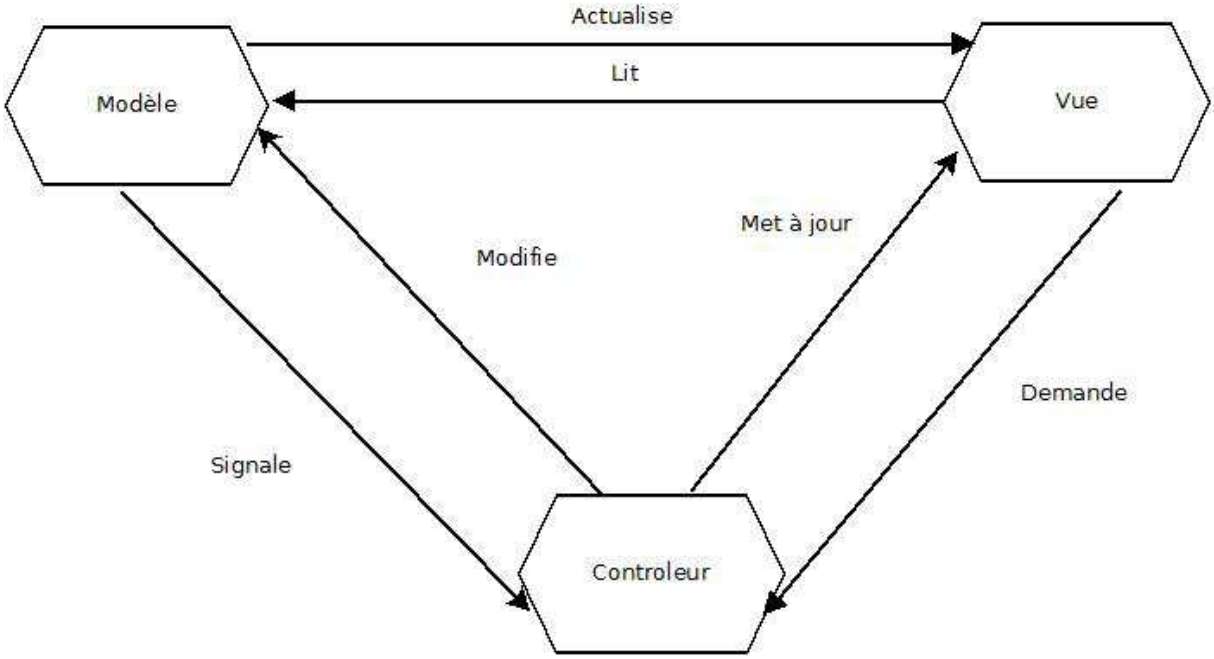
De cette question naît la notion de vue. On appelle vue une représentation d'une partie du programme.

Nous allons donc créer une vue principale regroupant quatre cadres (Frame) et les gouttières (parties séparant les cadres).

La première étape à atteindre est la mise en place de cet espace visuel.

Nous devons également chercher les paramètres appelés à changer (Largeur d'un cadre, position d'un cadre, …).Ces valeurs seront stockées dans le modèle. On appelle modèle l'ensemble des données du programme.

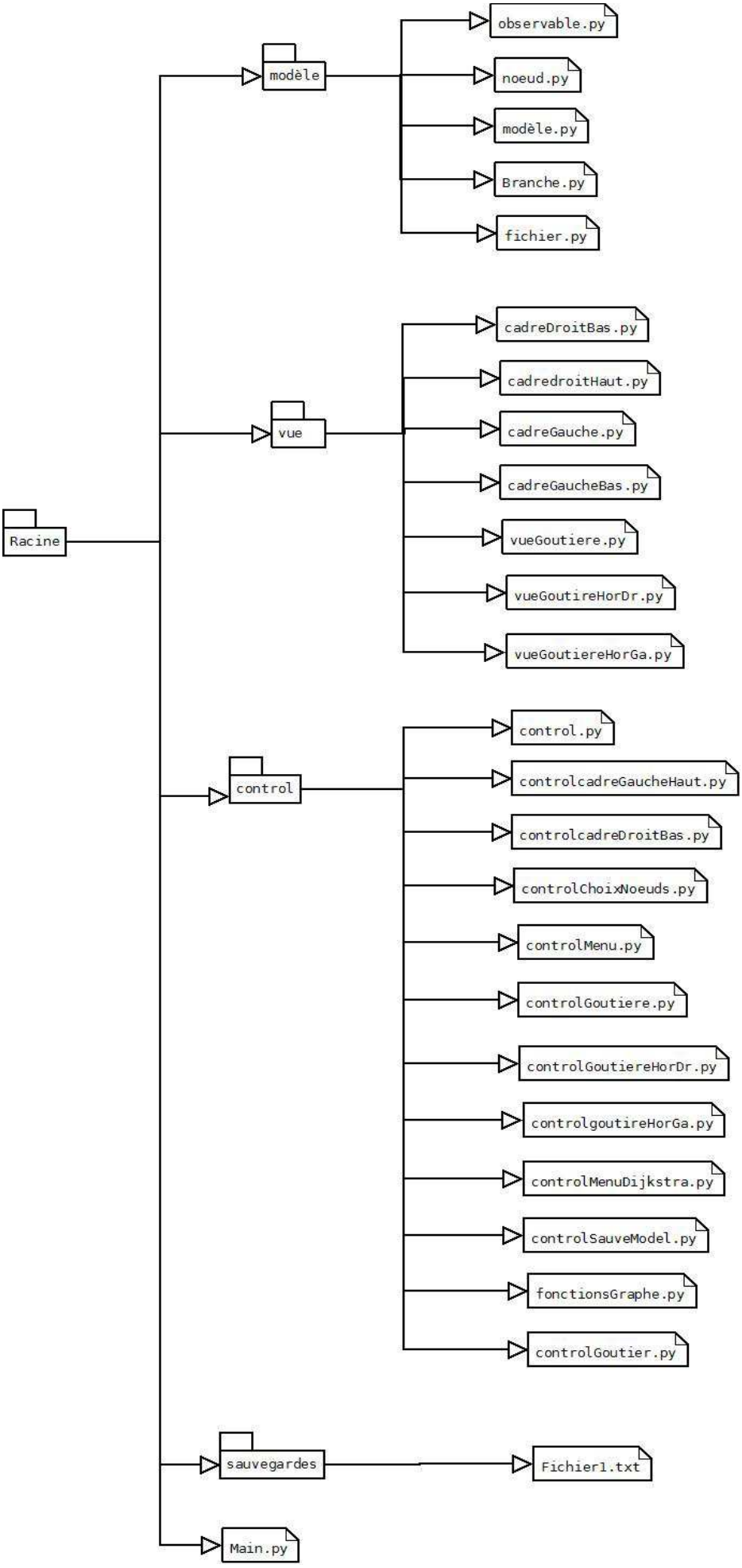
Afin de rendre le tout dynamique, créons des contrôleurs. On appelle contrôleur un programme qui écoute un événement. Si l'événement se produit, alors il réalise un diagnostic et demande éventuellement au modèle de modifier certaines de ses propriétés. Le modèle demandera alors à la vue de s'actualiser.




## b) Structure de l'arborescence

Au lieu de n'avoir qu'un seul fichier comprenant toutes les lignes de code d'un programme, la structure MVC permet en outre de le scinder en trois parties ce qui facilite la recherche de bogue lors de sa conception et facilite d'éventuelles évolutions.

Dans notre cas, nous avons choisi un découpage de notre programme de la manière suivante :




**Nota** : Le fichier main.py sera l'exécutable.

## c) Le modèle vue contrôleur

La classe modèle **hérite** de la classe Observable :
c'est-à-dire tous les attributs et les méthodes de la classe Observable appartiennent à la classe modèle.

```
#!/usr/bin/python
#-*- coding: UTF-8  -*-

class Observable:
    def __init__(self):
        self.listeObservable={}
    def ajoutObservable(self,obj):
        self.listeObservable[obj]=1
    def actualise(self):
        for loop in self.listeObservable:
            loop.adapte(self)
```

Cette classe est primordiale : inutile de la reconstruire pour chaque projet. Chaque fois que le modèle change, l'appel de la méthode actualise permet d'adapter toutes les vues.

Le contrôleur peut, après un événement, demandé aux modèles de modifier plusieurs de ces attributs. Ce dernier demandera après chacun des changements une adaptation des vues.

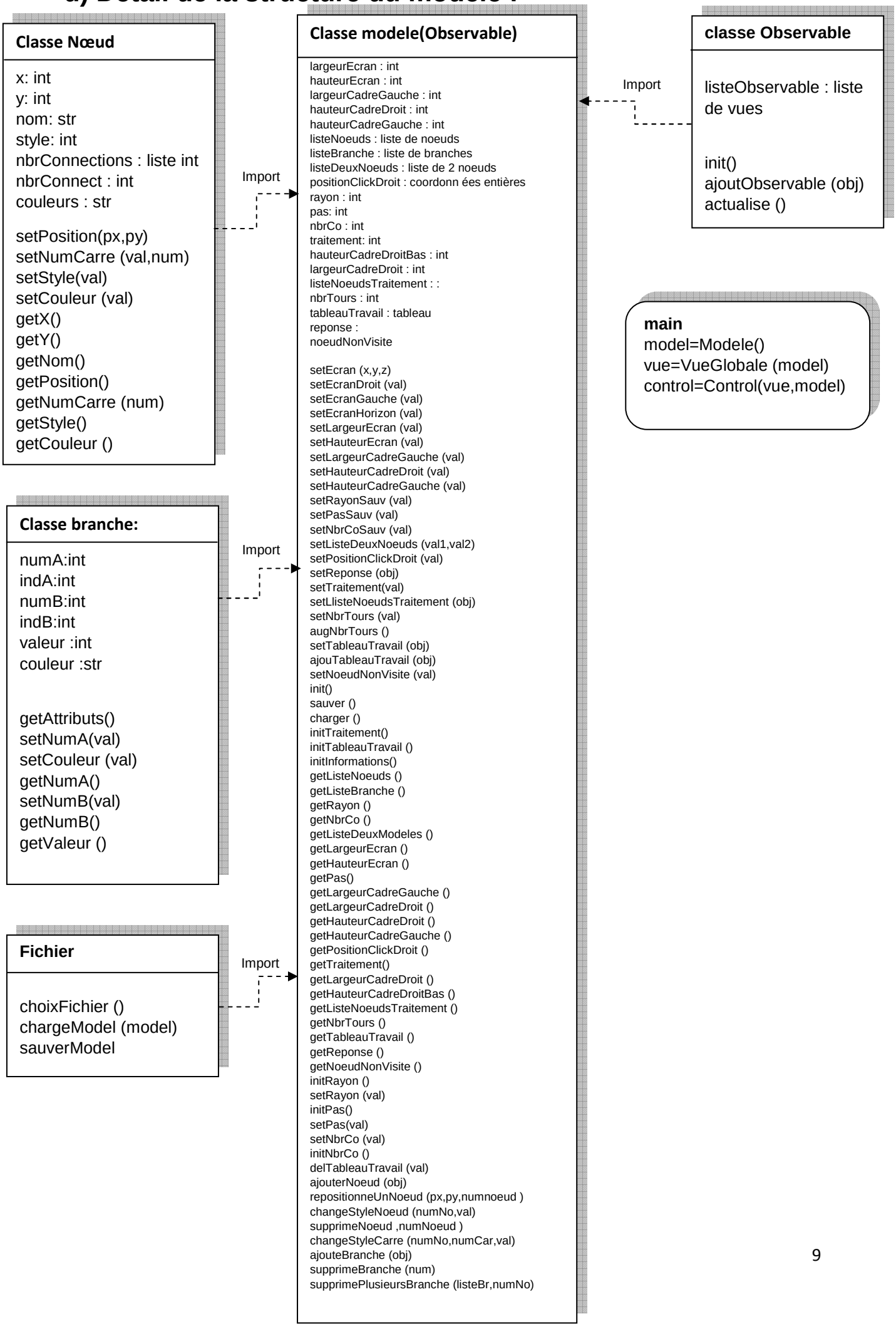
Les nœuds et les branches sont représentés par des classes. Attention lors de l'importation, ne pas oublier que l'url relative est positionnée au niveau du fichier main.py. Pour importer le fichier nœud.py, il faudra inscrire dans le code model.noeud.py. Facilité de stockage avec python : création d'une liste vide et ajout à l'aide de la méthode append de l'objet liste.

Concernant les classes, nous respectons la propriété **d'encapsulation :** les attributs des objets ne sont modifiables que par des méthodes de la classe. En python, ce n'est pas une obligation : pas de véritable variable privée. L'utilisation de deux ''underscores'' génère l'apparition d'un ''attributeError'' lors de l'accès direct à la variable.

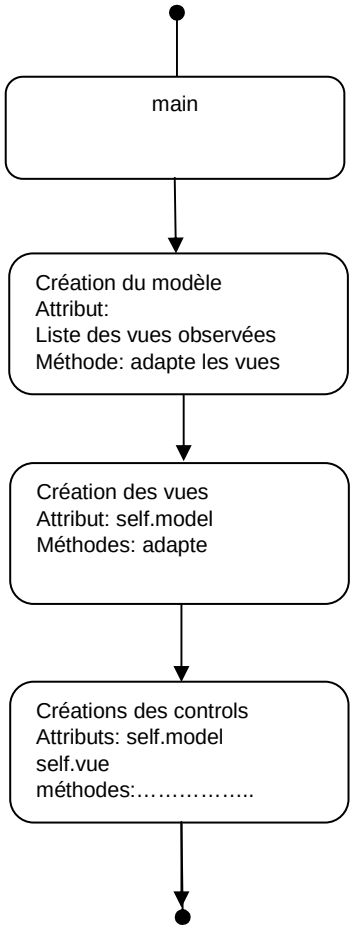
Par respect pour ce mode de programmation et pour un soucis de sécurité, nous n'enfreidrons pas ce principe.
Pour chaque attribut de la classe, nous crérons donc deux méthodes :

```
class Noeud:
    def __init__(self):
        self.attribut=4
    def getAttribut(self):
        """ renvoie la valeur de l'attribut """
        return self.attribut
    def setAttribut(self,valAttribut):
        """ permet de changer la valeur de l'attribut """
        self.attribut=valAttribut
```

# d) Détail de la structure du modèle :



## e) Détail du lancement de l’application :



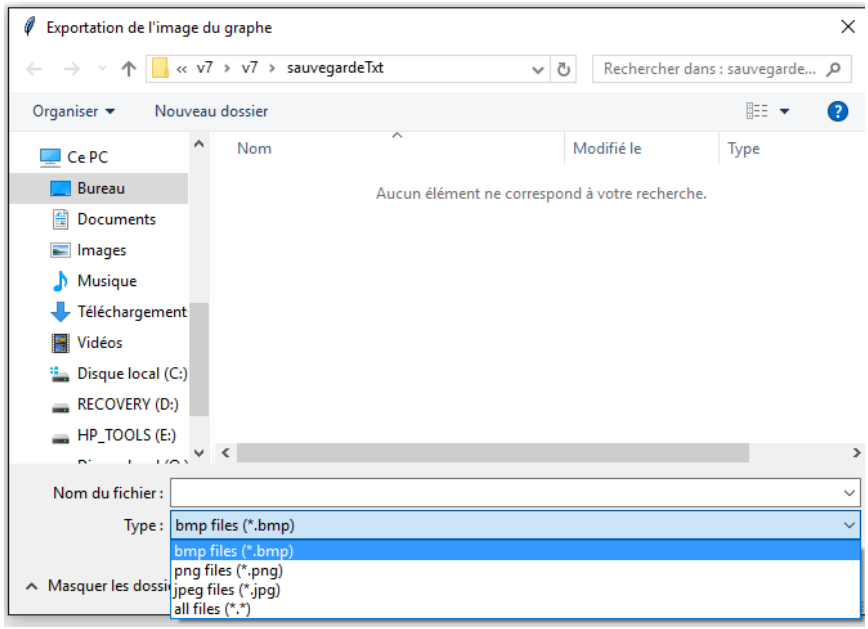
# IV. Problèmes rencontrés et améliorations possibles :

Les problèmes rencontrés ont été nombreux dont voici les plus pertinents avec pour certains les solutions envisagées :

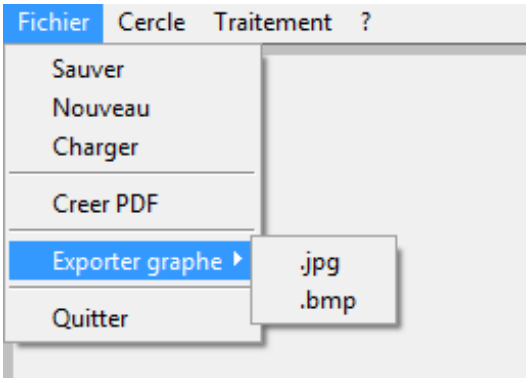
- La gestion de la sauvegarde sous la forme d'un fichier image du graphe a nécessité l'installation de la bibliothèque PIL, or celle-ci est venue interférée avec le module image de la bibliothèque Tkinter.

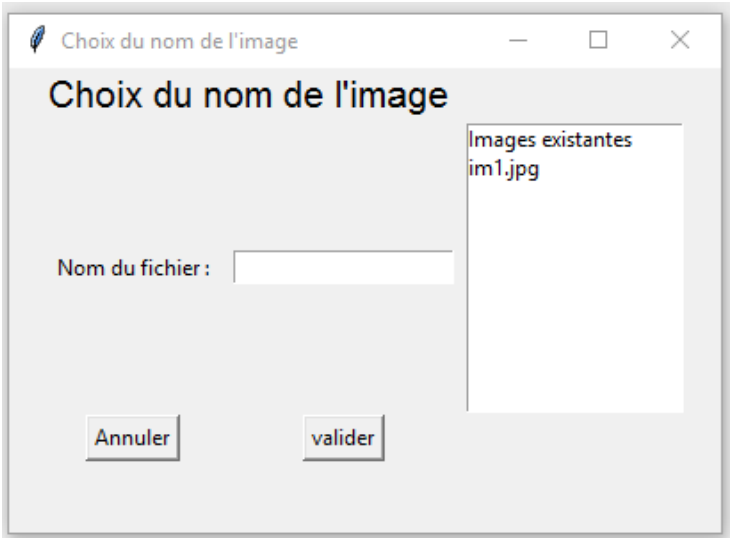
- La boite de dialogue askfilename  propose un menu déroulant permettant de sélectionner le type d'extension possible du fichier image, or ce paramètre n'est pas récupérable :

(fen1.filename =  filedialog.asksaveasfilename(initialdir = "/",title = "Exportation de l'image du graphe",filetypes = (("bmp files","*.bmp"),("png files","*.png"),("jpeg files","*.jpg"),("all files","*.*")))




La solution pour contourner ce problème a été de créer sous menu où l'utilisateur choisi une extension du fichier image parmi celles proposées qui ouvre une boite de dialogue.



- Le code n’est complètement pas optimisé : certains attributs sont dépendants. Il faut penser lors du changement de l’un d’entre eux à bien réévaluer les autres.

***Exemple*** : Largeur gauche et largeur droit

- Certaines parties du logiciel ne peuvent être testées que très tard dans le développement. Il est impossible de tester l’algorithme de Dijkstra sans un graphe. Pour tester l’algorithme, il faudrait disposer d’une batterie de graphes avec solutions. Difficiles à créer.

- Certaines actions peuvent provoquer plusieurs changements successifs du modèle. Après chacune d’entre elles, les vues s’actualisent. Ce déroulement pourrait entraîner des lenteurs d’exécutions. La solution proposée serait de reprendre chaque action possible afin de réaliser toutes les modifications du modèle dans un premier temps et après seulement actualiser les vues.

# V. Annexes :

## a) Les bibliothèques utilisées

1. **Tkinter** (Bibliothèque graphique)

Permet par l'utilisation de Widgets de rendre la programmation l'interface Homme/machine plus facile et plus agréable visuellement.

2. **Tkinter.font**

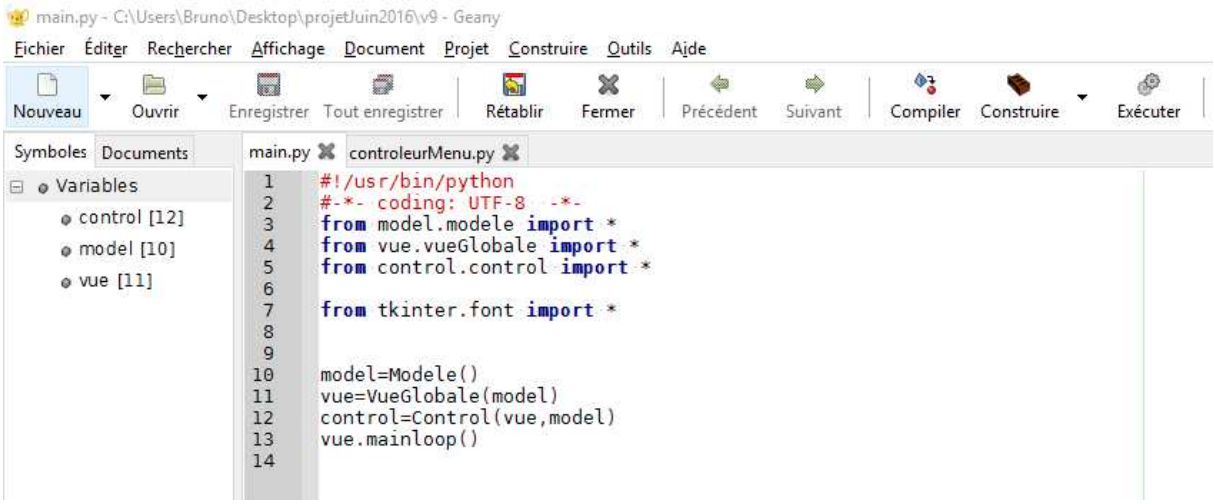
Permet de changer le style de typographie, la taille de la police, gras, italique...

3. **Pil**

Permet de faire du traitement d'image : chargement, couper/coller…

## b) Logiciel

Nous avons utilisé le logiciel de programmation **Geany** à la place d'**IDLE** car il permet la manipulation de plusieurs fichiers simultanément, grâce à l'usage d'onglets.



# VI. conclusion :

Lors de la conception, Il faut être très méthodique  car La quantité de dossiers et de fichiers devient très vite ingérable. Nous nous sommes servis d'un listing des méthodes du modèle et de la cartographie de la structure pour avancer dans le projet. Les méthodes doivent même être complètement identifiées : quel type de paramètre et quel type de réponse.

*Exemple* : ajouterNoeud(v1,p1,v2,v2,val) : le paramètre v1 est un entier( indice du nœud dans le tableau listeNoeuds) ou un objet ville. Avec Python, il faut faire très attention car les variables ne sont pas typées. Une erreur ne sera pas immédiatement relevée.

Les cours dispensés par M David Langlois et Mme Isabelle Rennesson nous ont permis de mieux appréhender la programmation orientée objet et la structure MVC rendant les programmes de notre application plus lisibles et plus facilement modifiables.

Par manque de temps, notre projet n'exploite pas les bases de données ni le traitement d'images vus en deuxième année, mais ces derniers nous seront très utiles lors de nos futurs enseignements en ISN.